Easy Campus Stay Assignment 2 Windows installation and run guide

Group 22 | Updated 31 October 2026 | Docker Desktop is already installed

Use this guide to move your existing Python and PHP application into Docker, run a booking, collect evidence and prepare the submission. Complete one numbered section at a time. Commands are for Windows PowerShell, not Command Prompt or a Linux terminal.

Required work	Optional extension
Dockerfiles, Docker Compose, working booking, restart and a genuine deployment incident. Core allocation: 9 marks.	Minikube deployment with a working booking, plus Inventory scaling and pod replacement. Available allocation: up to 3 marks.

Your working folder is C:\Projects\campusstay_assignment2. Keep this spelling everywhere. A repository name, an extracted ZIP folder and a local checkout folder can be different names.

Docker runs Python 3.12, PHP 8.3 with Apache, and MariaDB 10.11 inside Linux containers. Host Python 3.12 is used for the configuration and import helpers. XAMPP is not used by this deployment.

Read in this order

Sections 1-6: check Docker and Git, then put the deployment files into the cloned project. Sections 7-15: configure, run, test, restart and diagnose Compose. Sections 16-19: optional Minikube. Sections 20-22: final evidence, daily operation and troubleshooting.

The report is limited to five main pages excluding appendices. The institution recommends a 6-8 minute demo. Use the separate recording kit for screen changes and simple speaking lines.

Commands are also in docs/ASSIGNMENT2_WINDOWS_GUIDE.md. Copy a complete code block. A backtick at a line end continues the same PowerShell command; the >> prompt is normal while it waits for the next line.

1 Check your installed Docker Desktop

1. Open Docker Desktop from the Windows Start menu. Wait until its engine is running. Save unrelated work and close unused applications on an 8 GB laptop.

2. Open a normal Windows PowerShell window. Run these commands one at a time.

```
docker version
docker compose version
docker info --format '{{.OSType}}'
wsl --status
```

Expected: docker version shows both Client and Server. Compose prints a version. The operating-system check prints linux.

3. In Docker Desktop Settings, check the WSL 2 engine option when using the WSL backend. If the tray menu offers Switch to Linux containers, choose it. If it offers Switch to Windows containers, Linux containers are already selected.

4. Run the small Docker smoke check. Internet access is needed for its first image download.

```
docker run --rm hello-world
```

Expected: The container prints Hello from Docker and exits. It is a one-time check, not an Easy Campus Stay service.

If Docker is not ready

A daemon or named-pipe connection error means the Server is unavailable. Start Docker Desktop and wait, then repeat docker version. A WSL update prompt can be handled with wsl --update; restart when Windows requests it.

Check winver and the current official Windows requirements if Docker reports an unsupported system. An older Windows 10 build may require an OS update or another supported PC. Do not reinstall a working Docker Desktop just to follow this guide.

Official requirements: https://docs.docker.com/desktop/setup/install/windows-install/

2 Check Git and host Python

1. Run the following in PowerShell. You do not need to activate the old Assignment 1 virtual environment.

```
git --version
py -3.12 --version
```

Expected: Git prints its version. Python prints Python 3.12.x. Your earlier error already showed that Python 3.12 can run.

If a command is missing, install the matching tool and reopen PowerShell. Use https://git-scm.com/downloads/win and https://www.python.org/downloads/windows/.

2. If Ambuj or another member clones a private repository, they first accept the collaborator invitation while signed into the invited GitHub account. Each person uses their own account.

Already cloned on your PC

3. Use your existing checkout. Run:

```
Set-Location 'C:\Projects\campusstay_assignment2'
Get-Location
git status
git branch --show-current
git remote -v
```

Expected: The current path is C:\Projects\campusstay_assignment2. Git recognises a repository and shows its actual branch and remote.

If the working tree is clean and you want the latest commits from its tracked branch, run git pull --ff-only before creating your Assignment 2 branch. Resolve any reported Git issue before importing files.

Do not clone again into this same folder. Continue to section 3. If you have not cloned yet, use the next page.

3 Use the correct project folder

Only if you have not cloned yet

Skip this clone block when C:\Projects\campusstay_assignment2 is already your repository.

```
New-Item -ItemType Directory -Force 'C:\Projects'
Set-Location 'C:\Projects'
git clone `
  https://github.com/2025tm93137-cyber/EasyCampusStay---Room-Booking-System---microservices.git `
  campusstay_assignment2
Set-Location 'C:\Projects\campusstay_assignment2'
```

The final clone argument creates campusstay_assignment2. Do not enter easycampusstay_assignment2 afterward unless you deliberately created or renamed that folder.

Check why configure.py was missing

```
Get-ChildItem -Name
Test-Path '.\docker-compose.yml'
Test-Path '.\deploy\configure.py'
Get-ChildItem -Path . -Filter configure.py -Recurse -File |
  Select-Object -ExpandProperty FullName
```

Expected: Both Test-Path commands must return True before deployment. A scripts\configure.py result is the older local configuration helper; it is different from deploy\configure.py.

If the deployment file is in a nested project folder, enter the parent folder of deploy and docker-compose.yml. If it is absent, complete sections 4-6 to import the supplied deployment files. Python cannot create a script that is missing.

Even if both files already exist, use the preview step to check which deployment revision you have. The importer skips identical files. It never imports a private .env or replaces business-rule source files.

4 Extract the updated Assignment 2 package

1. Download Easy_Campus_Stay_Assignment2_Complete.zip supplied with these guides. Keep the downloaded ZIP outside your Git checkout.

2. In File Explorer, right-click the ZIP and choose Extract All. Set the extraction destination to C:\Projects\EasyCampusStay_A2_Package. Use a new empty destination; do not extract over an older package.

3. Open the extracted folder. It contains a top-level Easy_Campus_Stay_Assignment2 folder. Inside it you should see deploy, k8s, services, ui and docker-compose.yml.

4. Return to PowerShell and define these two variables. Run this block again if you open a new terminal before import.

```
$A2Package = 'C:\Projects\EasyCampusStay_A2_Package'
$A2Source = Join-Path $A2Package 'Easy_Campus_Stay_Assignment2'
Test-Path "$A2Source\deploy\import_assignment2.py"
Test-Path "$A2Source\deploy\configure.py"
```

Expected: Both checks return True. If either returns False, inspect the extracted folder name. An extra nested directory changes the path.

5. Keep the package separate from C:\Projects\campusstay_assignment2. The package is a source of reviewed deployment additions; your cloned folder retains Git history and genuine member changes.

The ZIP includes a full baseline for a separate test, but do not copy its entire services or ui folders over newer team work. The importer copies a defined list of deployment files and checks the small existing-file patches.

Repository availability and current branch contents depend on what your team has pushed. A successful Git clone downloads those committed files; it does not automatically include a separately supplied ZIP.

5 Preview the import before changing the checkout

1. Enter the cloned project and inspect current work.

```
Set-Location 'C:\Projects\campusstay_assignment2'
git status
git diff
```

2. Save and commit any genuine unfinished changes before importing. Choose only the files you reviewed; do not use git add . for this step. The importer requires a clean working tree when applying changes.

3. Create a new Assignment 2 branch. If this branch already exists, use git switch assignment2/docker-deployment instead of the creation command.

```
git switch -c assignment2/docker-deployment
```

4. Run the preview using the helper inside the extracted package. Preview does not write to your checkout.

```
py -3.12 "$A2Source\deploy\import_assignment2.py" `
  --target 'C:\Projects\campusstay_assignment2'
```

Preview label	Meaning
NEW	Deployment file will be added.
SAME	Identical deployment file will be kept.
REVIEW	A deployment file already exists with different content. Review it first.
PATCH APPLY	A small existing-file patch is compatible.
PATCH ALREADY	That patch is already present.
PATCH MANUAL	Current source differs; merge the documented change manually first.

The existing-file patches affect ui/bootstrap.php, the Payment and Notification logging helpers, and .gitignore. They support environment-only service URLs, approved Docker DNS names and container logging. They do not change booking business rules.

6 Apply the reviewed deployment additions

When the preview has no REVIEW or PATCH MANUAL rows

```
py -3.12 "$A2Source\deploy\import_assignment2.py" `
  --target 'C:\Projects\campusstay_assignment2' --apply
```

When REVIEW rows appear

Open the named deployment files in the package and checkout and compare them. Only after reviewing those replacements, use this explicit option. The helper backs up changed existing files outside the repository.

```
py -3.12 "$A2Source\deploy\import_assignment2.py" `
  --target 'C:\Projects\campusstay_assignment2' `
  --apply --replace-reviewed
```

The backup location is printed, under your user profile in EasyCampusStay_A2_Backups. The helper checks all file hashes and compatible patches before writing. A MANUAL patch or a dirty working tree stops the import before it changes files.

When PATCH MANUAL appears

Open PATCH_NOTES.md and deploy/patches/assignment2-existing-files.patch in the extracted package. Compare the small changes with your current files. Preserve newer member changes, merge the needed environment/allow-list/logging changes, test and commit that genuine merge, then preview again. Do not replace whole service source folders.

1. After a successful import, verify the project root and inspect the real changes.

```
Test-Path '.\deploy\configure.py'
Test-Path '.\docker-compose.yml'
git status
git diff --stat
git diff
```

Expected: Both checks return True. Review all changes before starting Docker. The helper has not committed or pushed anything.

If all deployment files were already identical, there may be nothing new to import. Continue using the current checked files. README_ASSIGNMENT2.md contains the deployment instructions without replacing your existing root README.

7 Create private Docker configuration

1. Stop the old Assignment 1 launcher or individual service terminals with Ctrl+C. Do not start scripts/run.py or the old PHP launchers alongside Docker. XAMPP can stay installed; you can stop it to free memory.

2. From the cloned project root, run:

```
Set-Location 'C:\Projects\campusstay_assignment2'
py -3.12 .\deploy\configure.py
```

Expected: The helper reports that a private root .env was created. There is no password prompt. It generates five random passwords without printing them.

If .env already exists, the helper keeps it. Retain a valid Assignment 2 .env when reusing an initialized database volume. If it belongs to another setup, review it privately before the first Docker start; do not regenerate passwords for an existing database volume.

3. Check that Git ignores private configuration and that Compose can read the required values.

```
git check-ignore .env
docker compose config --quiet
```

Expected: The first prints .env. The second finishes without an error or a resolved configuration dump.

The root .env is for Compose. The original services have local configuration files for Assignment 1, but the Docker runtime receives environment variables. These private files are excluded from the Docker build.

Use .env.example for public placeholder names. Do not commit passwords, show .env during the video, or run plain docker compose config on a shared screen because its resolved output includes credentials.

You do not need the old offline wheels, a new host venv, or a host Composer installation for this Docker workflow. Docker installs the application runtimes and pinned dependencies during image builds.

8 Build and start in Terminal A

1. Keep Docker Desktop running. In PowerShell Terminal A, enter the project folder and start the complete deployment.

```
Set-Location 'C:\Projects\campusstay_assignment2'
docker compose up --build
```

2. Wait for the build and startup output. The first build downloads base images and dependencies and can take several minutes. Do not open another launcher while waiting.

Expected: MariaDB becomes healthy. db-init provisions the schemas and exits with code 0. Inventory, Payment, Notification, Reservation and UI then become healthy.

Container role	What happens
db	One persistent MariaDB engine.
db-init	Runs database/account setup once and exits successfully.
inventory / reservation	Separate Python API containers.
payment / notification	Separate PHP and Apache mock API containers.
ui	PHP browser interface and API proxy.
tools	Optional test container; it runs only when explicitly requested.

Six containers keep running: db, four APIs and ui. db-init Exited (0) is correct. Seven project Dockerfiles create five application images and two support images; MariaDB is a downloaded upstream image.

Leave Terminal A open. It displays application logs. If a build fails, save the full first error and use the troubleshooting section before continuing.

The Docker Desktop Containers screen also shows the easy-campus-stay-a2 project. Expand it to inspect the same containers; the PowerShell output remains the main reproducible evidence.

9 Check each service in Terminal B

1. Open a second normal PowerShell window called Terminal B. All commands below run here while Terminal A remains open.

```
Set-Location 'C:\Projects\campusstay_assignment2'
docker compose ps -a
docker compose images
```

Expected: db and the five application containers are running and healthy. db-init shows Exited (0).

2. Check every public health endpoint individually.

```
Invoke-RestMethod http://127.0.0.1:8101/health
Invoke-RestMethod http://127.0.0.1:8102/health
Invoke-RestMethod http://127.0.0.1:8103/health
Invoke-RestMethod http://127.0.0.1:8104/health
Invoke-RestMethod http://127.0.0.1:8180/health
```

Expected: Each response reports status ok and its service name: inventory, reservation, payment, notification or ui.

3. If needed, inspect targeted logs.

```
docker compose logs --tail=80 db db-init
docker compose logs --tail=80 inventory reservation payment notification ui
```

An API root such as http://127.0.0.1:8101/ may return Not Found because no home page is defined there. Use /health for an API check, and port 8180 for the booking page.

Host ports 8101-8104 and 8180 map to container port 8080. Containers call each other using names such as http://inventory:8080. The database stays inside the Docker network; no Windows database port is published.

An API health endpoint checks its own database access. The UI /health checks the UI process. A successful complete booking is still needed to prove inter-service communication.

10 Make a normal booking in the browser

1. Open http://127.0.0.1:8180/. Go to Service tests, refresh the four service cards, then click Reset to normal. Return to Book a room.

2. Keep a future check-in date. Click Add sample rooms ONCE. Wait until the new room type is selected.

Expected: Five of five rooms are available, with a price of INR 2500 for one night. Adding sample rooms again creates another room type; it does not refill this one.

3. Keep a synthetic guest such as demo-guest-01 and guest@example.test. Do not enter real personal or payment information.

4. Click Confirm booking ONCE and wait. Use Refresh this booking to check the same booking afterward.

Expected: Your booking shows CONFIRMED, a booking ID, INR 2500, demo payment successful and demo notification sent. Availability becomes four of five.

5. Copy the full booking ID into your private demo notes. Also note its room type, date and remaining count. You will retrieve this same booking after restarting Docker.

If the selected room is sold out, select another room with stock for this date or add new sample rooms. If there are no rooms for the date, use the date where stock was created or add sample rooms for your selected date.

If the response could not be read, the interface can offer Retry same booking. This reuses the original request key. Clicking Confirm booking again normally begins another booking.

The browser UI calls service APIs through its PHP proxy. It does not read any service database. Payment and Notification are demonstrations: no real charge or email is made.

11 Run the Docker workflow checks

1. Keep the deployment running. In Terminal B, create the evidence folders and run the tools container.

```
New-Item -ItemType Directory -Force evidence\assignment2\compose
docker compose --profile test run --rm --build tools 2>&1 |
  Tee-Object evidence\assignment2\compose\workflow-console.txt
$A2ChecksExit = $LASTEXITCODE
if ($A2ChecksExit -ne 0) { throw 'Checks failed. Read the saved output.' }
```

Expected: The original booking flow, concurrency, idempotency and saga scripts pass. Twelve peer-database access attempts are denied.

Check	Correct result
Main flow	Reserve, demo payment, confirm and retry a failed notification.
Concurrency	20 different requests, capacity 5: five HTTP 201 and fifteen HTTP 409.
Idempotency	Same key/details replay one result; changed details get 409; simultaneous repeats have one effect.
Saga compensation	A declined payment cancels the booking and releases the held room once.
Isolation	Each of four service accounts is denied access to the three other databases: 12 denials.

2. Open the summary and the evidence directory.

```
notepad evidence\assignment2\workflow_summary.json
explorer .\evidence
```

Expected: The summary result is PASS and every named check passes. Examine actual output rather than assuming success.

These scripts create their own synthetic rooms. A sold-out room in the UI does not prevent them. The evidence folder contains separate test groups and historical bookings; the whole database is not expected to contain only one booking.

For Assignment 2 use this tools-container command. scripts/run_all.py starts local service processes and is intended for the earlier local deployment.

12 Understand replay and failure in the UI

1. After a normal booking in the current loaded page, open Service tests and click Replay original request.

Expected: The same original HTTP status/body and booking ID return. A normal successful original response is HTTP 201. Availability does not decrease again.

2. Click Test changed-payload conflict.

Expected: HTTP 409 is expected because a different guest reference is sent with the same key. This deliberate rejection is a passing behaviour.

3. Select Declined payment, click Go to booking, and make one NEW booking.

Expected: The booking becomes CANCELLED. The held room is released and availability returns to its value before this attempt.

4. Select Successful payment and notification Fail once, then retry. Make another new booking and inspect its notification.

Expected: The booking is CONFIRMED and notification is SENT with attempts = 2.

5. Click Reset to normal when finished.

Refreshing the whole browser page clears only the temporary replay request in that page. Bookings remain in the database and can be retrieved by ID. Looking up a booking does not restore its original browser request key.

These controls explain the business behaviour from Assignment 1. In the Assignment 2 video, prioritise deployment, running containers, one booking, troubleshooting and restart. Save the complete challenge evidence for inspection.

13 Restart and prove data persistence

1. Before stopping, keep the confirmed booking ID, room/date and remaining stock from section 10.

2. In Terminal B, stop and recreate the deployment while keeping its database volume.

```
docker compose down
docker compose up -d
docker compose ps -a
```

3. Wait for healthy services. Reopen http://127.0.0.1:8180/. Expand Already have a booking ID? and paste the earlier ID.

Expected: The same booking still exists and is CONFIRMED. Check the same room/date to confirm its earlier stock was retained.

Compose stores database data in the database_data named volume. down without -v removes the containers and network but retains that volume. A new successful booking alone does not prove the earlier booking survived.

4. Optionally repeat the tools command after restart to verify the complete workflow again. It creates fresh test data.

```
docker compose --profile test run --rm --build tools
```

Keep the same private .env when reusing the volume. Changing a MariaDB password environment variable does not rewrite a password already stored in an initialized database.

For normal daily pauses, docker compose stop followed later by docker compose start keeps the existing containers. The down/up demonstration is stronger evidence that containers can be recreated with retained database data.

Do not add -v during the persistence demonstration. That option deletes the project database volume.

14 Record a genuine deployment incident

1. Read docs/TROUBLESHOOTING_WORKSHEET.md. When your Docker deployment actually fails, record the first symptom, time, failing command or UI action, diagnosis, change and successful retest.

The missing deploy/configure.py error is a real setup issue, but it happened before Docker ran. The institution asks for a deployment-related problem in the containerized system; that setup error alone does not complete Task 3.

2. Start diagnosis with safe, targeted commands.

```
docker compose ps -a
docker compose logs --tail=100 db db-init
docker compose logs --tail=100 reservation ui
```

3. For a suspected Inventory URL or service-discovery problem, check only that value and DNS lookup.

```
docker compose exec reservation python -c `
  "import os; print(os.environ['INVENTORY_URL'])"
docker compose exec reservation python -c `
  "import socket; print(socket.gethostbyname('inventory'))"
```

Expected: In the supplied Compose configuration the Inventory URL is http://inventory:8080. DNS resolves inventory to a container-network address.

4. Fix the actual cause, recreate or rebuild the affected service when required, then repeat the same failed workflow. Save evidence of the successful result.

Possible genuine causes include a failed image build, bad service URL, occupied host port, database authentication error or unhealthy dependency. Explain the issue you actually observed.

The earlier UI environment/allow-list defect was verified with native PHP regression checks. It is not proof of an incident inside your Docker deployment. Do not replace your team incident with a claim that has not been observed.

Record the learning in plain words: what failed, why the diagnosis proved the cause, what fixed it, and what check would catch it next time.

15 Practise a labelled configuration fault

This optional exercise deliberately introduces a bad URL. Use it for understanding; label it as an injected fault. Task 3 still calls for a genuine deployment problem unless the instructor accepts disclosed fault injection.

1. Begin with a healthy Compose deployment. In Terminal B, run:

```
docker compose -f docker-compose.yml `
  -f deploy/troubleshooting/localhost.override.yml up `
  -d --no-deps --force-recreate reservation
```

2. Make a NEW booking in the UI and keep its ID. Inspect Reservation logs and INVENTORY_URL using section 14.

Expected: Reservation points to its own localhost:8101, where Inventory is not running. The booking may remain pending with HTTP 202 while recovery retries. This is not a confirmed booking.

3. Restore the correct base configuration by recreating Reservation without the override.

```
docker compose up -d --no-deps --force-recreate reservation
docker compose logs --tail=80 reservation
```

4. Wait for health, then refresh the SAME booking. If the workflow was already cancelled, create another booking and state that it is a new request.

Expected: A recoverable pending booking reaches CONFIRMED. Validate the full deployment using the tools container afterward.

Inside a container, localhost refers to that same container. Use the Compose service name and internal port to reach another container. The Windows browser still uses localhost and the published host port.

16 Optional Minikube preparation

Complete the core Compose work first. The selected optional route is Minikube deployment with a successful booking, plus Inventory scaling and replacement after deletion. This can address the three available advanced marks if demonstrated successfully.

1. Save Compose evidence before switching, then stop Compose to release memory and port 8180. Keep Docker Desktop running.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Capture-Compose.ps1
docker compose down
```

2. Install kubectl and Minikube only if missing. If winget is available, run the following and verify the publisher in each installer.

```
winget install --id Kubernetes.kubectl -e
winget install --id Kubernetes.minikube -e
```

3. Reopen PowerShell and return to the project.

```
Set-Location 'C:\Projects\campusstay_assignment2'
kubectl version --client
minikube version
```

Expected: Both commands print versions. Docker version still shows a working Linux server.

Official instructions: https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/ and https://minikube.sigs.k8s.io/docs/start/.

The supplied helper requests two CPUs and 3072 MB for Minikube. On an 8 GB PC, run Compose and Minikube separately and close unused applications. If Windows/Docker cannot provide the resources, use a supported larger machine for the optional demonstration.

All cluster helper commands explicitly use the minikube context. They do not deploy to an unrelated configured cluster.

17 Build and deploy to Minikube

1. From the project root, run the deployment helper. Its execution-policy override applies only to this PowerShell process.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Deploy-Minikube.ps1
```

It builds the project images, downloads MariaDB, starts Minikube with the Docker driver, loads images, creates the namespace and an in-memory Secret, deploys persistent MariaDB, runs db-init and waits for the five application Deployments. Finally it creates a diagnostic tools Pod.

Expected: The helper ends without a failed command and reports that the deployment is ready. Startup can take several minutes.

2. Inspect cluster resources and useful diagnostics.

```
kubectl --context=minikube -n easy-campus-stay get pods,jobs,services,pvc
kubectl --context=minikube -n easy-campus-stay logs job/db-init
kubectl --context=minikube -n easy-campus-stay get events `
  --sort-by=.lastTimestamp
```

Expected: Application pods are ready. The db-init Job is Complete. The database persistent volume claim is Bound.

Kubernetes and Compose have separate database volumes. Old Compose bookings do not automatically appear in Minikube. Use a new synthetic room and booking for this environment.

The Secret is generated from your private .env in memory. Do not display the Secret data or an environment dump while screen sharing. A Kubernetes Secret representation is not encryption by itself.

If a wait times out, save its actual output and inspect the affected pod or Job. Do not repeatedly delete the namespace to retry a single failing pod.

18 Open the Kubernetes UI and run checks

1. Open a dedicated PowerShell Terminal C at the same project root. Start port forwarding and leave this window running.

```
kubectl --context=minikube -n easy-campus-stay `
  port-forward service/ui 8180:8080
```

Expected: The terminal reports forwarding from 127.0.0.1:8180 to port 8080. Compose must be stopped so that port 8180 is free.

2. Open http://127.0.0.1:8180/. Add NEW sample rooms and make a normal booking.

Expected: The Kubernetes booking is CONFIRMED, demo payment and notification succeed, and four of five rooms remain. Showing Running pods alone is insufficient.

3. In Terminal B, run the cluster workflow checks.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Check-Minikube.ps1
```

Expected: The helper returns successfully and saves genuine cluster output under evidence\assignment2\minikube.

4. Inspect the copied summary and console output.

```
notepad evidence\assignment2\minikube\assignment2\workflow_summary.json
notepad evidence\assignment2\minikube\workflow-console.txt
```

The nested assignment2 folder comes from copying the tools container evidence tree. Original challenge folders are copied alongside it.

If the diagnostic Pod has expired, delete only deployment-tools, reapply k8s/40-tools.yml, wait for Ready and rerun the helper. Leave the namespace and database volume intact.

19 Demonstrate scaling and pod replacement

1. Keep the Kubernetes UI port-forward running in Terminal C. In Terminal B, run:

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Demonstrate-SelfHealing.ps1
kubectl --context=minikube -n easy-campus-stay get pods -l app=inventory
```

The helper asks for three Inventory replicas, records pod names and UIDs, deletes one Inventory pod, then waits for three Ready replicas again. A UID uniquely identifies a pod; a replacement has a different UID.

Expected: It reports three ready replicas restored.
evidence\assignment2\minikube\self-healing.json contains result PASS and readyReplicas 3. The deleted UID is absent from the after snapshot.

2. Inspect the evidence and return to the UI for another NEW normal booking.

```
notepad evidence\assignment2\minikube\self-healing.json
```

Expected: The booking works after replacement. This proves more than a healthy-looking pod list.

3. Repeat the cluster checks to save post-recovery workflow evidence.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Check-Minikube.ps1
```

This demonstrates manual scaling and Kubernetes Deployment reconciliation after a pod deletion. It does not demonstrate HPA autoscaling, recovery from every outage, or a highly available database. MariaDB remains a single replica.

Use the actual before/after JSON and working booking in the video. Keep the optional section out of your completion claims if the cluster demonstration has not succeeded.

20 Save evidence and complete Git submission

1. For Compose evidence while Compose is running, use the helper below. If you are currently on Minikube, keep its separate evidence and capture Compose during the Compose phase.

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File deploy\windows\Capture-Compose.ps1
```

2. Review evidence, your actual incident worksheet, contributions and AI disclosure. Add real Windows/Docker versions, results, commit references, repository access and video link to the editable report.

3. Open the existing root README.md and add a link to README_ASSIGNMENT2.md so faculty can find the deployment steps. Keep the earlier README content. Then inspect and stage only reviewed files. These are example paths.

```
notepad README.md
git status
git diff
git add docker-compose.yml .dockerignore .env.example .gitattributes
git add deploy k8s docs README.md README_ASSIGNMENT2.md
git add services/inventory/Dockerfile services/reservation/Dockerfile
git add services/payment/Dockerfile services/notification/Dockerfile
git add ui/Dockerfile ui/bootstrap.php
git add services/payment/src/platform.php services/notification/src/platform.php
git add .gitignore
git diff --staged --stat
git diff --staged
```

4. After reviewing the staged content, make your genuine commit and push your branch.

```
git commit -m "Add reviewed Assignment 2 container deployment"
git push -u origin assignment2/docker-deployment
```

Open a pull request on GitHub, have another member review it, and merge after validation. Each member should make and explain genuine changes. Find their real commit references with git log --oneline or the repository Commits page.

Review safe evidence separately before staging selected output paths. Do not commit private configuration or replace pending results with invented output. The baseline Linux reference files do not prove a Windows Docker run.

Follow the recording kit. Keep the report at five main pages excluding appendices, and the demo approximately 6-8 minutes. Verify faculty access to the repository and video.

21 Stop and resume on another day

Stop Compose while retaining data

```
Set-Location 'C:\Projects\campusstay_assignment2'
docker compose down
```

Resume Compose

Start Docker Desktop, wait for its engine, enter the same checkout with the retained .env, then:

```
docker compose up -d
docker compose ps -a
```

Expected: Wait for health before opening http://127.0.0.1:8180/. Rebuild with docker compose up -d --build after source or image changes.

Stop Minikube after the optional demo

Press Ctrl+C in Terminal C to stop port forwarding. In Terminal B run:

```
minikube stop
```

This retains Minikube state for a later practice. Do not delete the namespace or volumes for routine shutdown.

Resume the existing Minikube deployment

```
minikube start --driver=docker --cpus=2 --memory=3072
kubectl --context=minikube -n easy-campus-stay get pods
```

If configuration or images changed, follow the full deployment helper again. Otherwise wait for readiness and restart the Terminal C port-forward. Keep Compose stopped when using the same browser port for Kubernetes.

22 Troubleshooting and final checks

Observed error	What to check first
Cannot find project path	Use campusstay_assignment2, matching the final clone argument.
Cannot open deploy/configure.py	Check project root; extract package and complete preview/import.
Import says working tree must be clean	Save and commit genuine work first; inspect git status.
Import says PATCH MANUAL	Merge only the documented patch changes; preserve newer source.
Docker daemon / named-pipe error	Start Docker Desktop and wait; docker version needs a Server.
Port already allocated	Stop Assignment 1 launchers and the other deployment using that port.
db-init Exited (0)	Expected setup completion; inspect other service health.
API root returns Not Found	Use /health; the booking UI is on port 8180.
Database access denied after config changes	Check retained .env versus initialized volume; do not reset data casually.
Image build / package download fails	Save the first error; check Internet, proxy and Docker logs.
Minikube insufficient memory	Stop Compose and unused apps, or use a supported larger PC.
Kubernetes UI unavailable	Check Terminal C port-forward and pod readiness; Compose must be stopped.

Before submission: all required services run; a booking works; an earlier booking survives Compose recreation; the genuine incident has diagnosis/fix/retest; optional work has actual cluster evidence if claimed; contributions and AI use are accurate; links are accessible.

Official references checked for this revision: Docker Desktop Windows installation, Docker Compose startup order and networking, Kubernetes probes, Minikube start and Docker driver. Full URLs are in README_ASSIGNMENT2.md.